

HANDS-ON WORKSHOP · SELF-LEARNING EDITION

Async Processing with Python

Build a responsive banking API: from your first coroutine to a tested, production-shaped notification pipeline.

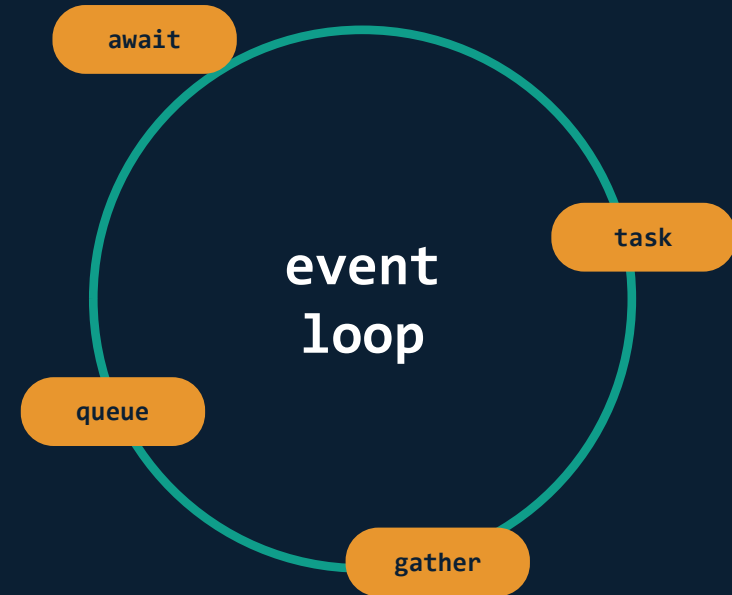
asyncio

FastAPI

httpx2

pytest-asyncio

Python 3.11+



Dev Team Onboarding · FIL Gurgaon · Trainer: Abinash Kumar Mishra

How to use this deck on your own



Every topic follows the same loop. Do not skip the "run it" step: async is learned by watching timings.



1. Read

CONCEPT slide: the idea + a small picture



2. Run

LAB slide: open the folder, run starter.py, do the TODOs



3. Compare

Check your output against EXPECTED OUTPUT



4. Peek

Still stuck? Open solution.py - read, then retype

Slide tags you will see

Tag	Meaning
CONCEPT	New idea, explained with a banking example
LAB	Hands-on: folder, steps, expected output
PITFALL	A real bug juniors hit in production code
CHECKPOINT	Quick quiz - answers are in the Appendix



Timing lines like [2.00s] are your best friend. Every lab prints them so you can SEE what ran in parallel.

What you will be able to do by the end



Explain

when async helps (I/O-bound) and when it does not (CPU-bound)



Write

coroutines and run them concurrently with `gather` and `TaskGroup`



Spot & fix

blocking calls that freeze the event loop



Harden

calls with timeouts, retries with backoff and clean cancellation



Build

a producer/consumer pipeline with `Queue` and `Semaphore`



Ship

a FastAPI service with background notifications, plus async tests

Day plan: 9 modules, 8 labs, one capstone



Approx. 6.5 hours including breaks. Self-learners: 2 short sessions of ~3 hours each work well.

#	Module	Lab folder	Time
0	Setup & kit tour	00_setup	15 min
1	Why async? Event loop mental model	-	25 min
2	Coroutines: async def & await	lab01_first_coroutine	35 min
3	Running things concurrently	lab02_tasks_gather_taskgroup	40 min
4	The blocking trap	lab03_blocking_trap	35 min
5	Timeouts, retries, cancellation	lab04_timeouts_retries_errors	40 min
6	Queues & rate limiting	lab05_queue_semaphore	40 min
7	Async HTTP clients	lab06_async_http	35 min
8	Async in FastAPI	lab07_fastapi_banking	55 min
9	Capstone + testing async code	lab08_capstone	90 min

Step 0 - Set up your machine (do this before class)



Python 3.11 or newer (3.13 recommended) · VS Code with the Python extension · two terminals

WINDOWS

```
# Windows PowerShell
> cd async-python-lab-kit
> py -3.13 -m venv .venv
> .venv\Scripts\Activate.ps1
> python -m pip install -r requirements.txt
> python 00_setup/verify_setup.py
```

MACOS / LINUX

```
# macOS / Linux
$ cd async-python-lab-kit
$ python3 -m venv .venv
$ source .venv/bin/activate
$ python -m pip install -r requirements.txt
$ python 00_setup/verify_setup.py
```

requirements.txt

```
fastapi[standard]>=0.115 # + uvicorn + CLI
httpx>=2.13 # async HTTP client
pytest>=8.3
pytest-asyncio>=1.0 # async tests
```



Activate.ps1 blocked by policy? Use cmd and `.venv\Scripts\activate.bat`. Behind the proxy? `pip install --proxy http://host:port ...` or your team's internal mirror.

Done when `verify_setup.py` prints "All good - you are ready!"

Tour of the lab kit



Same structure everywhere: README (what to do) · starter.py (your work) · solution.py (reference)

FOLDER LAYOUT

```
async-python-lab-kit/
├── README.md  CHEATSHEET.md  TROUBLESHOOTING.md
├── requirements.txt
├── 00_setup/verify_setup.py
├── lab01_first_coroutine/      README, starter, solution
├── lab02_tasks_gather_taskgroup/
├── lab03_blocking_trap/
├── lab04_timeouts_retries_errors/
├── lab05_queue_semaphore/
├── lab06_async_http/          + local mock server
├── lab07_fastapi_banking/     + client, load test
├── lab08_capstone/
│   ├── starter/  app/ tests/  <- make 11 tests pass
│   └── solution/ app/ tests/
```

EXPECTED OUTPUT

```
[OK ] Python 3.13 (need 3.11+ ...)
[OK ] fastapi      0.1xx.x
[OK ] uvicorn      0.xx.x
[OK ] httpx2       2.xx.x
[OK ] pydantic     2.xx.x
[OK ] pytest       x.x.x
[OK ] pytest_asyncio 1.x.x
[OK ] asyncio: event loop works
```

All good - you are ready!



Always **cd** into the lab folder before running its files.

MODULE

01

Why async?

Concept only | 25 min

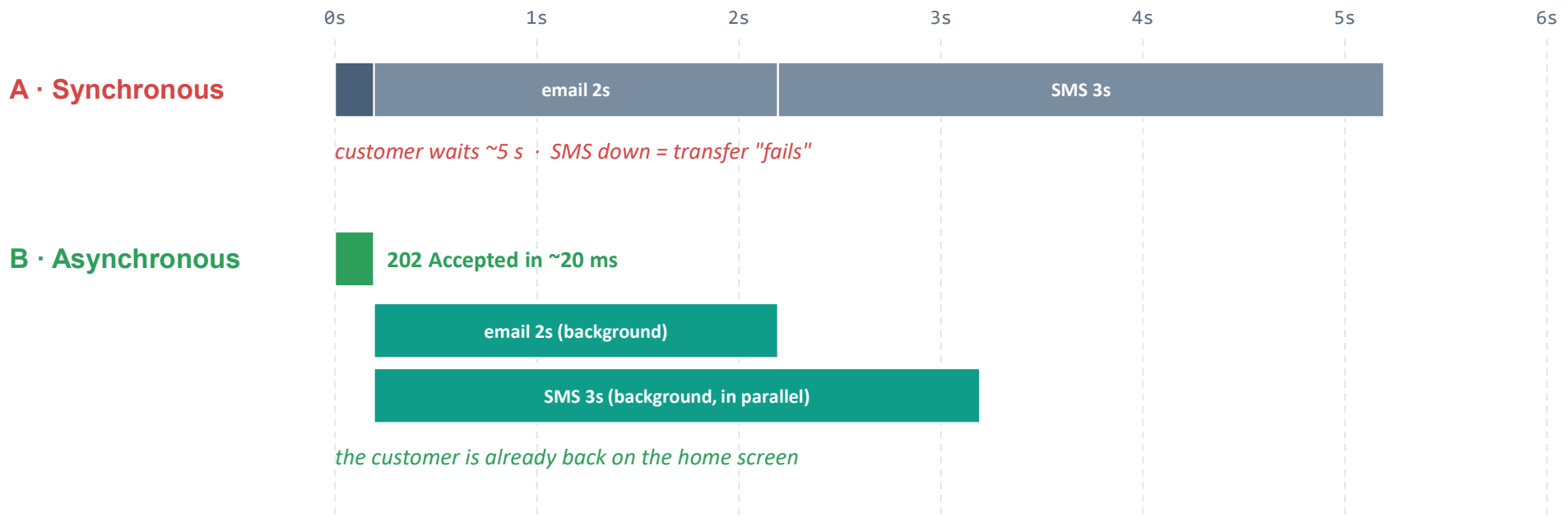
After this module you can

- ✓ Explain why a banking app should not wait for SMS/email
- ✓ Tell I/O-bound from CPU-bound work
- ✓ Describe what the event loop does

The story: a fund transfer



You tap "Send". What should the customer wait for?



Rule: do the **critical** part (move money) before responding; do the **non-critical, slow** part (notify) asynchronously. We build exactly this in Labs 7-8.

I/O-bound vs CPU-bound: pick the right tool



Async shines when your program mostly WAITS. It does not make calculations faster.



I/O-bound (waiting)

- HTTP calls to partner banks, SMS, email vendors
- Database queries, message queues, file reads
- CPU is idle ~99% of the time



CPU-bound (computing)

- Interest calculation over millions of rows
- PDF / image generation, encryption, ML inference
- CPU is busy 100% of the time

Tool	Best for	How many run at once	Python API
asyncio	lots of I/O (1000s of connections)	many tasks, 1 thread	async / await
threads	I/O with blocking (legacy) libraries	tens-hundreds of threads	asyncio.to_thread
processes	CPU-heavy work	= number of CPU cores	ProcessPoolExecutor

Concurrency = dealing with many things at once (one cook, many pans). Parallelism = doing many things at once (many cooks).

The event loop: one thread, tasks take turns



Cooperative multitasking: a task runs until it hits await, then politely gives the loop back.



Task A WAITING
await db.get()

Task B RUNNING
running now

Task C WAITING
await sms.send()

Task D READY
ready to resume

1

Loop picks a READY task
and runs it

2

Task runs until it hits await
e.g. waiting for a network reply

3

Task is parked; loop picks the next
the OS tells the loop when I/O is done

4

Waiting task becomes READY again
and resumes exactly where it paused



Consequence: if a task never awaits (e.g. `time.sleep(3)` or a heavy loop), NOBODY else runs. Module 4 is all about this.

Vocabulary you need (5 words)



coroutine

```
async def f(): ...
```

A function that can pause. Calling `f()` creates a coroutine object; it does NOT run yet.

await

```
x = await f()
```

Run `f` and pause me until it finishes. Only allowed inside `async def`.

event loop

```
asyncio.run(main())
```

The scheduler. `asyncio.run` creates it, runs `main()`, closes it. Call it once.

task

```
asyncio.create_task(f())
```

A coroutine scheduled on the loop to run concurrently with others.

awaitable

```
coroutine | Task | Future
```

Anything you can put after `await`.

MODULE

02

Coroutines: `async def` & `await`

Lab 1 | 35 min

After this module you can

- ✓ Write and run your first coroutine
- ✓ Explain why async code can still be slow
- ✓ Use `asyncio.gather` for real concurrency

Anatomy of an async program

Three keywords do all the work: `async def`, `await`, `asyncio.run`

```
anatomy.py
import asyncio

async def fetch_balance(account: str) -> int: # (1)
    print(f"-> asking core banking for {account}")
    await asyncio.sleep(2) # (2)
    return 5_000

async def main() -> None:
    balance = await fetch_balance("ACC-101") # (3)
    print("balance:", balance)

asyncio.run(main()) # (4)
```

- 1 **async def**
marks a function that may pause
- 2 **await**
pauses THIS coroutine; the loop is free to run others
- 3 **await a coroutine**
runs it and gives you its return value
- 4 **asyncio.run(main())**
the single entry point of the program



await only works inside **async def**. In a normal script you enter the async world exactly once, with **asyncio.run(...)**.

LAB 1 - Your first coroutine



Folder: lab01_first_coroutine · 20 minutes · Goal: 6 s -> 2 s

1

Run the baseline

python starter.py -> ~6 s, one account after another

2

Make it async (TODO 1-4)

async def · await asyncio.sleep(delay) · asyncio.run(main())

3

Run again. Still ~6 s!

awaiting one-by-one is still sequential

4

Use asyncio.gather (TODO 6)

all three requests start together -> ~2 s

5

Break it on purpose

remove one await and read the RuntimeWarning

the key change

```
b1, b2, b3 = await asyncio.gather(
    fetch_balance("ACC-101"),
    fetch_balance("ACC-102"),
    fetch_balance("ACC-103"),
)
```

EXPECTED OUTPUT (after step 4)

```
[ 0.00s] -> requesting balance for ACC-101
[ 0.00s] -> requesting balance for ACC-102
[ 0.00s] -> requesting balance for ACC-103
[ 2.00s] <- received balance for ACC-101
[ 2.00s] <- received balance for ACC-102
[ 2.00s] <- received balance for ACC-103
[ 2.00s] [concurrent] Total balance = 17800
```

Done when all three "->" lines show 0.00s and the total is ~2 s.

Lab 1 debrief: async def alone is NOT concurrency



await one after another (6 s)



asyncio.gather (2 s)



sequential

```
b1 = await fetch_balance("ACC-101")    # wait 2 s
b2 = await fetch_balance("ACC-102")    # then 2 s
b3 = await fetch_balance("ACC-103")    # then 2 s
```

concurrent

```
b1, b2, b3 = await asyncio.gather(
    fetch_balance("ACC-101"),
    fetch_balance("ACC-102"),
    fetch_balance("ACC-103"))    # all at once
```

Why it works

gather wraps each coroutine in a Task, so all three are on the loop at the same time. While one waits, the others wait too - in parallel.

Bonus fact

gather returns results in the order you passed them in - not the order they finished.

Pitfall: the forgotten await



The code "runs", no exception is raised, but nothing happened.

```
bug.py
async def send_sms(phone: str) -> None:
    await sms_gateway.send(phone, "Transfer done")

async def create_transfer(...):
    move_money(...)
    send_sms(customer.phone)      # BUG: no await -> SMS never sent
    return {"status": "ok"}
```

WHAT YOU SEE (easy to miss in logs)

```
RuntimeWarning: coroutine 'send_sms' was never awaited
    send_sms(customer.phone)
RuntimeWarning: Enable tracemalloc to get the object
allocation traceback
```



How to avoid it

- **Want the result now?**
- `await send_sms(...)`
- **Fire and forget?**
- `create_task(...)` and keep a reference (Module 3)
- **Enable linting:**
- Ruff / Pylance flag un-awaited coroutines
- **Treat RuntimeWarning as a failure in tests:**
- `pytest -W error::RuntimeWarning`

MODULE

03

Running things concurrently

Lab 2 | 40 min

After this module you can

- ✓ Choose between `gather`, `create_task` and `TaskGroup`
- ✓ Handle partial failures without crashing
- ✓ Read `ExceptionGroups` with `except*`

Three ways to run many coroutines



Scenario: end-of-day reconciliation - pull transaction counts from 5 branches

	<code>asyncio.gather</code>	<code>asyncio.create_task</code>	<code>asyncio.TaskGroup</code> (3.11+)
Shape	<code>await gather(a(), b())</code>	<pre>t = create_task(a()) ... await t</pre>	<pre>async with TaskGroup() as tg: tg.create_task(a())</pre>
Starts	when awaited	at the next await of the caller	at the next await inside the block
If one fails	exception raised, others keep running	you handle each task	siblings cancelled, <code>ExceptionGroup</code>
Partial success	<code>return_exceptions=True</code>	check each task	catch with <code>except*</code>
Use when	simple fan-out, want results in order	start now, collect later	default for new code



Default choice: `TaskGroup`. It guarantees no task outlives the block, so there are no orphaned background tasks and no lost errors.

TaskGroup and partial failure in code



```
lab02 / solution.py
async def run_with_taskgroup() -> list[int]:
    async with asyncio.TaskGroup() as tg:
        tasks = [tg.create_task(fetch_txn_count(b))
                  for b in HEALTHY]
        # here ALL tasks are done (or an error was raised)
    return [t.result() for t in tasks]

async def run_tolerant() -> None:
    branches = HEALTHY + ["PUNE"]          # PUNE is down
    results = await asyncio.gather(
        *(fetch_txn_count(b) for b in branches),
        return_exceptions=True)           # errors -> values
    for branch, res in zip(branches, results):
        if isinstance(res, Exception):
            print(f"{branch} FAILED: {res!r}")
```

```
except* (3.11+)
try:
    async with asyncio.TaskGroup() as tg:
        for b in HEALTHY + ["PUNE"]:
            tg.create_task(fetch_txn_count(b))
except* ConnectionError as eg:
    for exc in eg.exceptions:
        print("caught:", exc)
```

Read it like this

- TaskGroup = all or nothing
- gather(return_exceptions=True) = best effort
- except* matches exceptions inside an ExceptionGroup

LAB 2 - Tasks, gather() and TaskGroup



Folder: lab02_tasks_gather_taskgroup · 30 minutes

1

TODO A - gather

fetch 4 healthy branches; results in input order

2

TODO B - create_task

start tasks, print "preparing report...", then await them

3

TODO C - TaskGroup

same result using async with asyncio.TaskGroup()

4

TODO D - tolerant

add PUNE; return_exceptions=True; print which failed

5

Bonus

read taskgroup_failure_demo() in solution.py

EXPECTED OUTPUT

```
=== run_with_gather ===
[ 0.00s] -> GURGAON
[ 0.00s] -> NOIDA
[ 0.00s] -> DELHI
[ 0.00s] -> BENGALURU
[ 0.50s] <- DELHI
[ 1.00s] <- GURGAON
[ 1.50s] <- BENGALURU
[ 2.00s] <- NOIDA
result: [700, 500, 500, 900]
took 2.00s
...
PUNE      FAILED: ConnectionError('PUNE branch
server unreachable')
```

Done when A, B, C each take ~2.0 s (the slowest branch) and D reports PUNE without crashing.

Surprise from Lab 2: tasks start at the next await



Look at the order of lines in TODO B

```
tasks = [asyncio.create_task(fetch_txn_count(b)) for b in HEALTHY]
log("preparing report template...")    # printed FIRST
await asyncio.sleep(0.2)                # <- tasks actually start here
```

ACTUAL OUTPUT

```
[ 2.00s] preparing report template while branches respond...
[ 2.00s] -> GURGAON
[ 2.00s] -> NOIDA
[ 2.00s] -> DELHI
```



Why?

`create_task` only SCHEDULES the coroutine. The loop can run it only when the current coroutine gives control back - at its next `await`. One thread, taking turns.



Second trap - lost tasks: the loop keeps only a weak reference to tasks. `asyncio.create_task(send_sms())` with no variable can be garbage-collected mid-run. Keep a reference (list/set) or use a `TaskGroup`.

Checkpoint 1 - quick quiz

 CHECKPOINT

Answer on paper first. Answers: Appendix A.

Question 1

`fetch()` sleeps 1 s. How long does `"for i in range(3): await fetch()"` take? And `"await asyncio.gather(fetch(), fetch(), fetch())"`?

Question 2

What does `x = fetch()` (no `await`) put in `x`?

Question 3

Branch PUNE fails. Which call keeps the other branches' results: `gather(...)`, `gather(..., return_exceptions=True)` or `TaskGroup`?

Question 4

True or false: async code uses multiple CPU cores automatically.

MODULE

04

The blocking trap

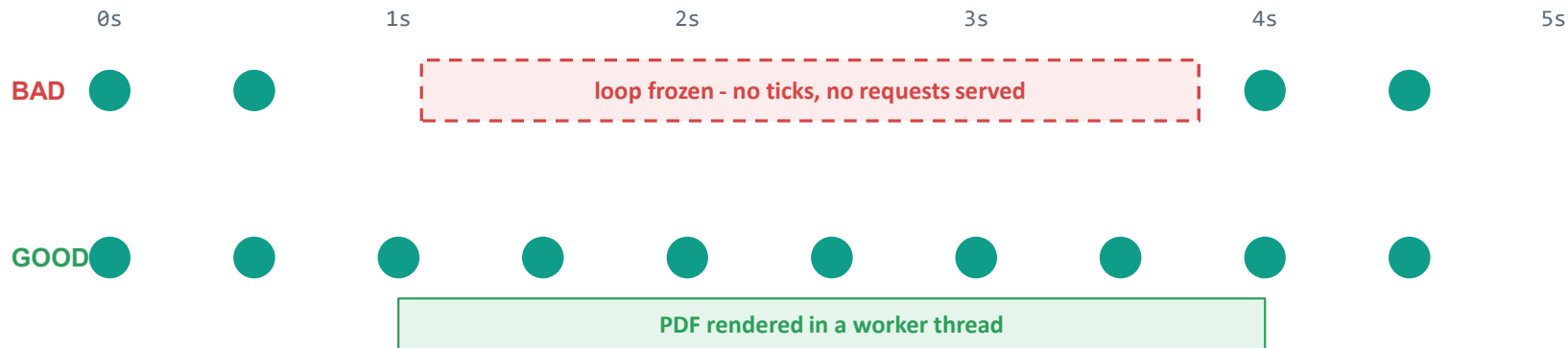
Lab 3 | 35 min

After this module you can

- ✓ Recognise calls that freeze the event loop
- ✓ Offload blocking I/O with `asyncio.to_thread`
- ✓ Offload CPU work to a process pool
- ✓ Use asyncio debug mode

One blocking call freezes everyone

A heartbeat task prints "tick" every 0.5 s. Then a PDF statement is generated with a blocking call.



BAD

```
async def render_pdf_bad(customer):
    path = blocking_render(customer) # sleeps 3 s
    return path
```

GOOD

```
async def render_pdf_good(customer):
    path = await asyncio.to_thread(
        blocking_render, customer)
    return path
```

In a web server "ticks" are other customers' requests. One blocking call = every user waits.

Blocking -> non-blocking replacements



Inside async def, anything that waits without "await" is suspicious.

Blocking (inside async def)	Replace with	Why
<code>time.sleep(2)</code>	<code>await asyncio.sleep(2)</code>	yields to the loop while waiting
<code>requests.get(url)</code>	<code>await client.get(url)</code>	httpx2.AsyncClient - async HTTP
sync DB driver (e.g. psycopg2)	async driver (asyncpg, psycopg 3 async, SQLAlchemy async)	native async I/O
legacy SDK with no async version	<code>await asyncio.to_thread(sdk.call, arg)</code>	runs in a worker thread
CPU-heavy function	<code>await loop.run_in_executor(process_pool, fn, arg)</code>	separate process = real parallelism
reading a huge file	<code>await asyncio.to_thread(path.read_bytes)</code>	disk I/O off the loop

```
loop = asyncio.get_running_loop()
with ProcessPoolExecutor() as pool:
    total = await loop.run_in_executor(pool, compute_interest, 15_000_000)
```



Windows/macOS: process pools need the `__main__` guard in the script.

LAB 3 - The Blocking Trap



Folder: lab03_blocking_trap · 25 minutes

1

Run starter.py

watch the ticks stop for 3 s

2

Turn on debug mode (TODO 1)

`asyncio.run(main(), debug=True)`

3

Fix with `to_thread` (TODO 2)

`await asyncio.to_thread(blocking_render, customer)`

4

Offload CPU work (TODO 3)

`ProcessPoolExecutor + run_in_executor`

5

Verify

ticks never stop in solution.py

DEBUG MODE WARNING (step 2)

```
Executing <Task pending name='Task-1'
  coro=<main() running at starter.py:53> ...>
  took 3.001 seconds
```

EXPECTED OUTPUT (solution)

```
[ 1.00s] PDF: start (in a worker thread)
[ 1.00s] tick
[ 1.50s] tick
[ 2.01s] tick
...
[ 4.00s] PDF: done -> statement_CUST-42.pdf
[ 4.01s] Interest: start (in a worker process)
[ 4.01s] tick
[ 4.90s] Interest: done -> 29999998
```

Tip: `PYTHONASYNCIODEBUG=1` enables the same checks without code changes.

MODULE

05

Timeouts, retries, cancellation

Lab 4 | 40 min

After this module you can

- ✓ Put a timeout on every external call
- ✓ Retry transient errors with exponential backoff
- ✓ Cancel tasks and clean up correctly

Three rules for calling unreliable services

The SMS gateway sometimes hangs, sometimes fails. Your service must not.



Rule 1 · Always a timeout

A hung vendor must not hang your worker forever.

```
async with asyncio.timeout(1.0):
    return await gateway.send(phone)
# -> raises TimeoutError
```



Rule 2 · Retry transient errors

Connection resets and 503s often succeed on retry. Back off exponentially.

```
delay = base * 2 ** (attempt - 1)
# 0.2s, 0.4s, 0.8s ...
```



Rule 3 · Respect cancellation

Clean up, then re-raise CanceledError. Never swallow it.

```
except asyncio.CancelledError:
    cleanup()
    raise
```



Only retry what is safe to repeat. Re-sending an SMS is fine. Re-debiting an account is NOT - payments need idempotency keys. Never retry validation errors (4xx).

Retry with exponential backoff

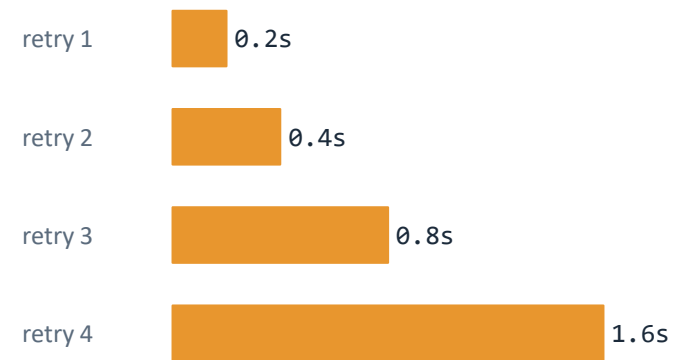


lab04 / solution.py

```
async def with_retry(make_call, attempts=3, base_delay=0.2):
    # make_call is a FACTORY: a coroutine can be awaited only once,
    # so every attempt needs a fresh one -> lambda: gateway.send(phone)
    for attempt in range(1, attempts + 1):
        try:
            return await make_call()
        except ConnectionError as exc:
            if attempt == attempts:
                raise # give up
            delay = base_delay * 2 ** (attempt - 1) # 0.2, 0.4, 0.8
            log(f"attempt {attempt} failed - retrying in {delay:.1f}s")
            await asyncio.sleep(delay)

result = await with_retry(lambda: gateway.send("+91-98100-00002"))
```

Wait before each retry



Production add-on: jitter (random +/-) so 1000 clients do not retry at the same instant. Libraries: tenacity, stamina.



Common bug: passing a coroutine object (`gateway.send(p)`) instead of a factory -> 2nd attempt fails with "cannot reuse already awaited coroutine".

LAB 4 - Timeouts, Retries, Cancellation

Folder: lab04_timeouts_retries_errors · 30 minutes

1

TODO A - timeout

wrap `hanging_gateway` in `async` with `asyncio.timeout(1.0)`; return "TIMEOUT"

2

TODO B - retry

implement `with_retry`; `FlakyGateway` fails twice then succeeds

3

TODO C - cancellation

catch `CancelledError`, log cleanup, re-raise

4

Experiment

change `FlakyGateway(failures=5)`: what happens? Why?

EXPECTED OUTPUT

```
=== A timeout ===
[ 1.00s] gateway too slow - gave up after 1 s
[ 1.00s] result: TIMEOUT

=== B retry ===
[ 1.10s] attempt 1 failed: gateway error on call 1 - retrying in 0.2s
[ 1.40s] attempt 2 failed: gateway error on call 2 - retrying in 0.4s
[ 1.90s] result: sent to +91-98100-00002 on call 3

=== C cancellation ===
[ 1.90s] sending to +91-98100-00003 ...
[ 2.41s] cancelled -> cleanup: mark notification as PENDING_RETRY
[ 2.41s] task reported as cancelled - correct!
```

Done when your output matches, including the timings.

MODULE

06

Queues & rate limiting

Lab 5 | 40 min

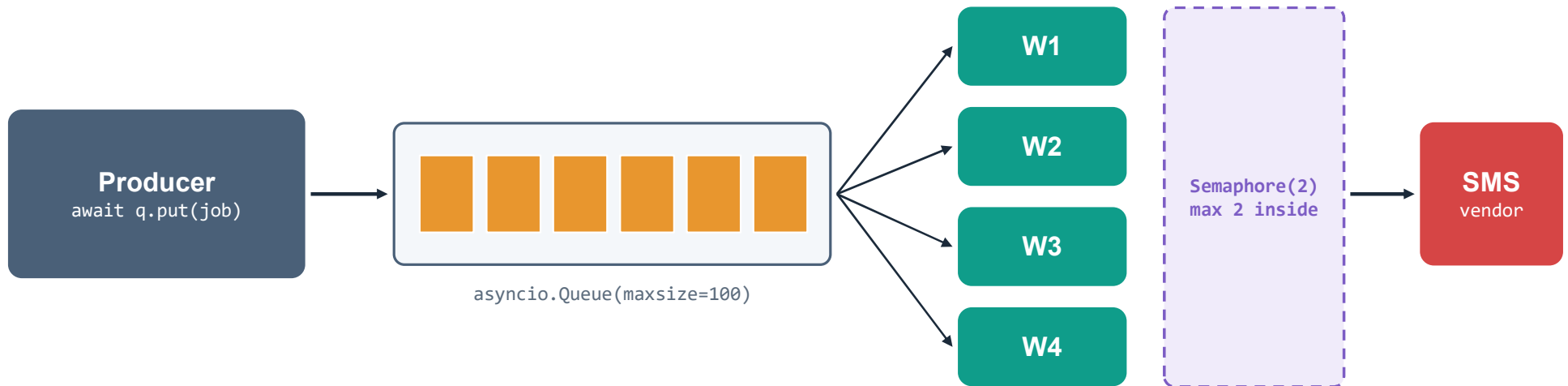
After this module you can

- ✓ Decouple producers and consumers with `asyncio.Queue`
- ✓ Cap concurrency with a Semaphore
- ✓ Shut down workers cleanly

Producer / consumer: the async backbone



12 transfers just happened; each needs an SMS. The vendor allows max 2 concurrent connections.



Decoupling

the API only puts a job and returns; workers do the slow part

Back-pressure

a bounded queue makes producers wait when workers fall behind

Rate limit

the semaphore protects the vendor, however many workers you run

The worker pattern (memorise this)



```
lab05 / solution.py

async def worker(name: str, queue: asyncio.Queue[Notification]) -> None:
    while True:
        n = await queue.get()          # wait (non-blocking) for work
        try:
            await send_sms(n, name)    # uses 'async with sms_limit:'
        except Exception as exc:      # one bad job must not kill us
            log(f"{name} failed T{n.transfer_id}: {exc!r}")
        finally:
            queue.task_done()          # ALWAYS, or join() hangs

async def main() -> None:
    queue: asyncio.Queue[Notification] = asyncio.Queue(maxsize=100)
    workers = [asyncio.create_task(worker(f"W{i}", queue))
               for i in range(1, 5)]
    await producer(queue)             # put 12 jobs
    await queue.join()                # every job got task_done()
    for w in workers:
        w.cancel()                    # workers loop forever
    await asyncio.gather(*workers, return_exceptions=True)
```



Semaphore

`sms_limit = asyncio.Semaphore(2)`

async with `sms_limit`:
 await `vendor.send(...)`



Checklist

- `task_done()` in finally
- `await queue.join()`
- cancel workers after
- bounded maxsize
- 3.13+: `queue.shutdown()`

LAB 5 - Queue + Semaphore



Folder: lab05_queue_semaphore · 30 minutes · Goal: 6 s -> 3 s, never more than 2 in flight

1

Run starter.py
sequential: ~6 s

2

Queue + producer (TODO 1-2)
put 12 Notification objects

3

Workers (TODO 3-5)
4 workers, join(), then cancel

4

Semaphore(2) (TODO 6)
watch in-flight never exceed 2

5

Experiment
fill in the table on the right

Workers	Semaphore	Your time	Why?
4	2		
4	4		
4	1		
1	4		
12	2		

EXPECTED OUTPUT (end)

```
[ 2.51s] W3 sending T11 (in-flight=1)
[ 2.51s] W4 sending T12 (in-flight=2)
[ 3.01s] all notifications sent; max in-flight = 2
```

Answers to the experiment table: Appendix A.

Checkpoint 2 - quick quiz

 CHECKPOINT

Answers: Appendix A.

Question 5

An async handler calls `requests.get()` which takes 2 s. 10 users call it at once. Roughly how long does the last user wait?

Question 6

Which API do you use to call a legacy, blocking SDK from async code without freezing the loop?

Question 7

Why must `with_retry` receive `lambda: send(p)` instead of `send(p)`?

Question 8

Your worker forgot `queue.task_done()`. What is the symptom?

MODULE

07

Async HTTP clients

Lab 6 | 35 min

After this module you can

- ✓ Call HTTP APIs concurrently with `httpx2.AsyncClient`
- ✓ Reuse one client (connection pool)
- ✓ Handle HTTP and network errors

httpx2.AsyncClient: the async way to call APIs



Scenario: fetch INR rates for 6 currencies from a partner bank

```
lab06 / solution.py
import asyncio
import httpx2

limit = asyncio.Semaphore(3)          # be polite to partners

async def get_rate(client: httpx2.AsyncClient, ccy: str) -> dict:
    async with limit:
        resp = await client.get(f"/fx/{ccy}")
        resp.raise_for_status()        # 4xx/5xx -> error
        return resp.json()

async def main() -> None:
    async with httpx2.AsyncClient(base_url=BASE, timeout=5.0) as client:
        rates = await asyncio.gather(
            *(get_rate(client, c) for c in CURRENCIES))
```



Rules

- ONE client per app - it holds the connection pool
- Always set timeout=
- `raise_for_status()` then catch `httpx2.HTTPStatusError`
- Network problems: `httpx2.RequestError`

httpx vs httpx2

httpx2 is the successor of httpx (same API, now used by FastAPI's `TestClient`). Projects on httpx 0.28 can use the same code with "import httpx".



requests is sync-only: never call it inside **async def**.

LAB 6 - Async HTTP against a local mock bank



Folder: lab06_async_http · 30 minutes · needs TWO terminals

TERMINAL 1

```
# Terminal 1 - Leave running
$ fastapi dev mock_bank_api.py --port 8001
# fallback: python -m uvicorn mock_bank_api:app --port 8001
# browse: http://127.0.0.1:8001/docs
```

TERMINAL 2

```
# Terminal 2
$ python client_sync.py      # baseline
$ python starter.py         # your work (TODO 1-5)
$ python solution.py        # reference
```

Version	Expected time
client_sync.py (for-loop)	~6 s
gather, no limit	~1 s
gather + Semaphore(3)	~2 s
get_rate("XYZ")	handled: 404 unknown currency XYZ



Why a local mock? Same latency for everyone, no proxy issues, no API keys.

Done when you get ~2 s with Semaphore(3) and the 404 is handled gracefully.

Stop a server with Ctrl+C. "Address already in use"? An old server is still running.

MODULE

08

Async in FastAPI

Lab 7 | 55 min

After this module you can

- ✓ Choose def vs async def for endpoints
- ✓ Return 202 fast and notify in the background
- ✓ Know when BackgroundTasks is not enough

def vs async def in FastAPI



FastAPI runs the two kinds of endpoints in different places.



async def + await

Runs ON the event loop. Perfect when everything slow inside is awaited (httpx2, async DB driver, asyncio.sleep).



plain def

Runs in a thread pool. Safe for blocking code (sync DB driver, legacy SDK). Limited by pool size.



async def + blocking

Runs on the loop AND blocks it: the whole server serves one request at a time. The worst option.

```
@app.get("/slow/async-blocking")
async def bad():
    time.sleep(1)           # freezes the server
```



Rule of thumb: use **async def** only if every slow call inside is awaited. Otherwise use plain **def**.

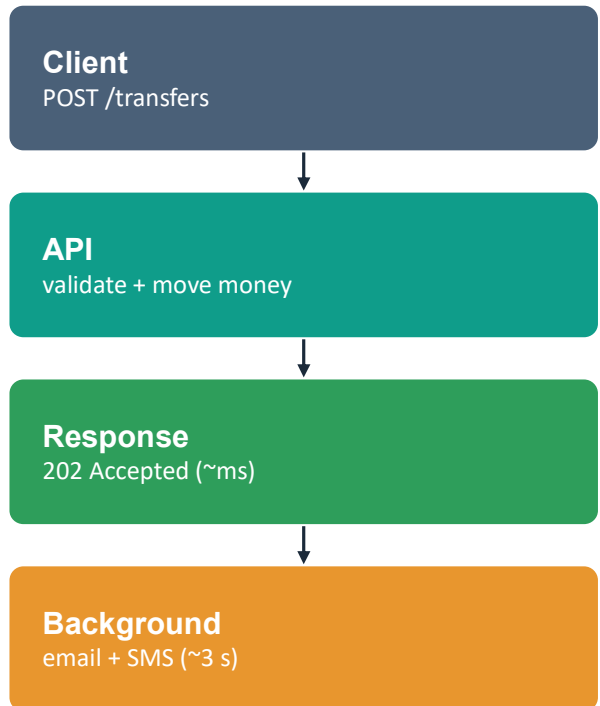
BackgroundTasks: respond first, notify after



lab07_fastapi_banking/main.py

```
main.py (simplified)
@app.post("/transfers", status_code=202)
async def create_transfer(req: TransferRequest,
                          background_tasks: BackgroundTasks):
    # 1. critical + fast: validate + move money BEFORE answering
    if ACCOUNTS[req.from_account] < req.amount:
        raise HTTPException(status_code=400, detail="insufficient funds")
    ACCOUNTS[req.from_account] -= req.amount
    ACCOUNTS[req.to_account] += req.amount
    transfer_id = f"TXN-{uuid.uuid4().hex[:8].upper()}"

    # 2. non-critical + slow: runs AFTER the response is sent
    background_tasks.add_task(send_notifications, transfer_id, req)
    return TransferAccepted(transfer_id=transfer_id,
                             status="ACCEPTED", message="...")
```



Pydantic validates the body: amount must be > 0, or FastAPI returns 422 automatically.

LAB 7 - FastAPI Mini Bank



Folder: lab07_fastapi_banking · 45 minutes · two terminals

Part A - BackgroundTasks

1

Terminal 1: fastapi dev main.py

open <http://127.0.0.1:8000/docs>

2

POST /transfers in Swagger UI

202 in a few ms; watch terminal 1

3

Terminal 2: python client.py

see 202, 400, 422 and the new balance

4

Break the rules

amount -5 (422) · unknown account (404)

client.py OUTPUT

```
202 {'transfer_id': 'TXN-7F653669', 'status': 'ACCEPTED',  
      'message': 'Transfer done. Notification ...'} in 0.008s  
400 {'detail': 'insufficient funds'}  
422 Input should be greater than 0  
{'account': 'ACC-101', 'balance': 48500.0, ...}
```

SERVER TERMINAL

```
14:32:35.734 [api] TXN-7F653669: money moved, responding  
14:32:35.735 [bg] TXN-7F653669: sending email + SMS ...  
14:32:38.736 [bg] TXN-7F653669: notifications sent
```



No curl needed. In Windows PowerShell `curl` is an alias of `Invoke-WebRequest` - use Swagger UI, `client.py` or `curl.exe`.

LAB 7 Part B - predict, then measure



python load_test.py fires 5 parallel requests at each endpoint (each sleeps 1 s). Predict first!

Endpoint	Code inside	Your prediction	Measured
/slow/async-blocking	async def + time.sleep(1)		
/slow/sync-def	def + time.sleep(1)		
/slow/async-nonblocking	async def + await asyncio.sleep(1)		

```
$ python load_test.py
```

Answers: Appendix A



Discuss after measuring

- Which endpoint would take down production under load?
- Why is plain def "fine" here, and what limits it at 1000 requests?
- What would you change in a code review?

When BackgroundTasks is not enough



Pick the lightest tool that meets your delivery guarantee.

BackgroundTasks

same process, after the response

Lost on restart · no retry · no status

Lab 7

In-process Queue + workers

lifespan-managed asyncio workers

Status tracking, retries, rate limits · still lost on crash

Lab 8

Broker + worker processes

Celery / RQ / Arq / Dramatiq + Redis or RabbitMQ; Kafka; SQS

Survives restarts · scales out · monitoring

Production



Money movement itself is never "background": it must be committed before you answer.

MODULE

09

Capstone: Mini Bank notification pipeline

Lab 8 | 90 min

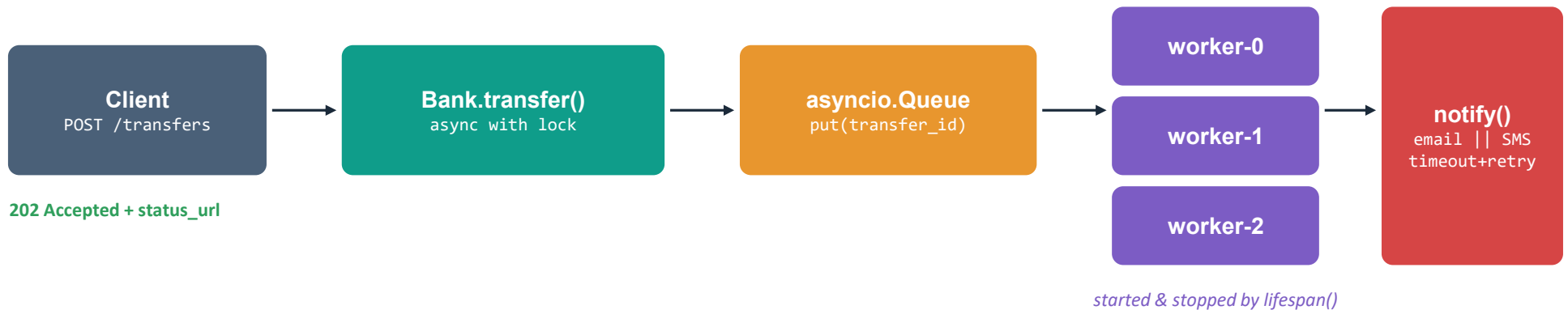
After this module you can

- ✓ Combine lock, queue, workers, retry and timeouts
- ✓ Manage workers with FastAPI lifespan
- ✓ Test async code with pytest-asyncio

Capstone architecture

💡 CONCEPT

lab08_capstone - everything from Labs 1-7 in one production-shaped service



GET /transfers/{id} -> notification_status



File	Responsibility
app/bank.py	accounts + async transfer with an asyncio.Lock
app/notifier.py	send_email / send_sms, with_retry, notify (parallel)
app/main.py	FastAPI app, lifespan, workers, endpoints

Yes, async code has race conditions too



Any await between "check" and "act" lets another task run in between.

starter/app/bank.py (BUG)

```
async def transfer(self, src, dst, amount):
    await self._db_roundtrip()      # read  <- others run here!
    if self.accounts[src].balance < amount:
        raise TransferError("insufficient funds")
    await self._db_roundtrip()      # write
    self.accounts[src].balance -= amount
```

FIX (TODO 1)

```
async with self._lock:             # check + act = atomic
    await self._db_roundtrip()
    if self.accounts[src].balance < amount:
        raise TransferError("insufficient funds")
    await self._db_roundtrip()
    self.accounts[src].balance -= amount
    self.accounts[dst].balance += amount
```



The failing test

10 concurrent transfers of 100 from an account holding 500. All 10 read "500" before anyone debits. Result: balance -500.

assert 10 == 5 (FAILED)



In real systems

Use the database: transactions with SELECT ... FOR UPDATE, or an atomic UPDATE ... WHERE balance >= amount.

Testing async code



pytest-asyncio for coroutines · FastAPI TestClient for the API (runs lifespan inside "with")

```
pytest.ini

[pytest]
asyncio_mode = auto    # run async tests
pythonpath = .
testpaths = tests
```

```
tests/test_bank.py

async def test_no_overdraft_under_concurrency():
    bank = small_bank()          # A has 500
    results = await asyncio.gather(
        *(bank.transfer("A", "B", 100)
          for _ in range(10)),
        return_exceptions=True)
    assert sum(r is None for r in results) == 5
    assert bank.accounts["A"].balance == 0
```

```
tests/test_api.py

from fastapi.testclient import TestClient
from app.main import app

def test_transfer_is_accepted_fast_and_notified_later():
    with TestClient(app) as client:    # runs lifespan
        t0 = time.perf_counter()
        r = client.post("/transfers", json={
            "from_account": "ACC-101",
            "to_account": "ACC-102", "amount": 1500})
        assert r.status_code == 202
        assert time.perf_counter() - t0 < 0.5 # no wait
        tid = r.json()["transfer_id"]
        body = wait_for_final_status(client, tid)
        assert body["notification_status"] == "SENT"
```



Tests set **NOTIFY_DELAY=0.05** in `confestest.py` so fake vendors are fast.
Make delays configurable in your own code too.

LAB 8 - Capstone: make 11 tests green



Folder: lab08_capstone/starter · 75 minutes · work in pairs

TERMINAL

```
$ cd lab08_capstone/starter
$ pytest -v          # start: 8 failed, 3 passed
$ pytest -v -x       # after each TODO
$ fastapi dev app/main.py # then open /docs
```

Order	TODO	File	Turns green
1	Lock the read-check-write	app/bank.py	no_overdraft test
2	with_retry: timeout + backoff	app/notifier.py	test_retry_* (3)
3	notify() in parallel (gather)	app/notifier.py	test_notify_* (2)
4	Start workers in lifespan (TODO 5)	app/main.py	(needed for API)
5	Worker loop (TODO 4)	app/main.py	test_api.py (2)

DONE WHEN

```
tests/test_api.py::test_transfer_is_accepted_fast_and_notified_later PASSED
tests/test_api.py::test_sms_failure_is_reported_not_crashed PASSED
tests/test_api.py::test_business_and_validation_errors PASSED
tests/test_bank.py::test_no_overdraft_under_concurrency PASSED
...
===== 11 passed in 1.9s =====
```

Hints: Lab 4 (retry), Lab 1/2 (gather), Lab 5 (worker loop). Reference: lab08_capstone/solution.

Lifespan: start and stop background workers safely



Replaces the deprecated `@app.on_event("startup"/"shutdown")`

```
solution/app/main.py

@asynccontextmanager
async def lifespan(app: FastAPI):
    # ---- startup ----
    app.state.bank = seed_bank()
    app.state.transfers = {}
    app.state.queue = asyncio.Queue(maxsize=1000)
    workers = [asyncio.create_task(notification_worker(f"w{i}", app))
               for i in range(WORKERS)]

    yield # <- app serves requests here
    # ---- shutdown: drain, then stop ----
    try:
        async with asyncio.timeout(5):
            await app.state.queue.join()
    except TimeoutError:
        log.warning("%d not sent", app.state.queue.qsize())
    for w in workers:
        w.cancel()
    await asyncio.gather(*workers, return_exceptions=True)

app = FastAPI(title="Mini Bank - Capstone", lifespan=lifespan)
```

1

Before yield

create shared state + start workers

2

yield

server is running

3

After yield

graceful drain with a timeout

4

Cancel + gather

no orphan tasks, clean exit



Keep references: the `workers` list.

Production checklist for async Python

 REFERENCE

Use it in code reviews

- ✓ No blocking calls inside `async def` (`time.sleep`, `requests`, sync DB drivers)
- ✓ Shared state changed across an `await` is protected (Lock / DB transaction)
- ✓ Every external call has a timeout
- ✓ Concurrency to vendors is capped (Semaphore)
- ✓ Retries only for transient errors, with backoff (+ jitter)
- ✓ One shared HTTP client per app
- ✓ `CancelledError` is never swallowed
- ✓ Background work that must survive restarts goes to a broker
- ✓ Every `create_task` has an owner (TaskGroup, list, or lifespan)
- ✓ Async tests exist for the happy path AND failures

Key takeaways



- 1 Async helps when you WAIT (I/O), not when you COMPUTE.
- 2 `async def` enables concurrency; `gather` / `TaskGroup` delivers it.
- 3 One blocking call freezes everyone - offload with `to_thread` or a process pool.
- 4 Timeouts, retries with backoff and clean cancellation make it production-ready.
- 5 Queue + workers + Semaphore = responsive API and polite vendors.
- 6 Respond fast (202), do slow work after - and test both paths.

Final self-assessment



You are ready when you can answer all of these without the slides. Answers: Appendix A.

S1

Explain to a colleague why a FastAPI `async def` endpoint calling `requests.get()` slows down the whole service.

S2

Write from memory: a coroutine that fetches 3 URLs concurrently with `httpx2` and a 2-second timeout.

S3

What is the difference between `gather(return_exceptions=True)` and `TaskGroup` when one task fails?

S4

Why does the capstone need an `asyncio.Lock` even though `asyncio` is single-threaded?

S5

Name two things `BackgroundTasks` cannot guarantee, and what you would use instead.

S6

What must a worker always do in `finally`, and what happens if it does not?

APPENDIX

Answers · Troubleshooting · Cheat sheet · Further reading · Trainer notes

Appendix A - Answers



Checkpoint 1

- Q1: 3 s sequential; 1 s with gather.
- Q2: a coroutine object - nothing ran (RuntimeWarning later).
- Q3: `gather(..., return_exceptions=True)`.
- Q4: False - one thread; use processes for CPU.

Checkpoint 2

- Q5: ~20 s - the loop is blocked, requests are served one by one.
- Q6: `await asyncio.to_thread(fn, ...)`.
- Q7: a coroutine can be awaited once; each retry needs a new one.
- Q8: `await queue.join()` never returns - hang.

Lab 5 table

- (4 workers, sem 2) 3 s · (4,4) 1.5 s · (4,1) 6 s
- (1,4) 6 s - one worker = one job at a time
- (12,2) 3 s - the semaphore is the bottleneck

Lab 7 Part B

- `async-blocking` ~5 s: loop blocked
- `sync-def` ~1 s: thread pool
- `async-nonblocking` ~1 s: awaits yield

Self-assessment

- S3: TaskGroup cancels siblings + ExceptionGroup; gather returns errors as values.
- S4: awaits between check and act let other tasks interleave.
- S5: survival across restarts, retries/status -> broker (Celery/Arq/Kafka/SQS).
- S6: `queue.task_done()`; otherwise `join()` hangs.

Appendix B - Troubleshooting



Full list: TROUBLESHOOTING.md in the kit

Symptom	Fix
python not recognized (Windows)	use <code>py -3.13 ...</code> or reinstall with "Add to PATH"
Activate.ps1 cannot be loaded	<code>cmd + .venv\Scripts\activate.bat</code> , or call <code>.venv\Scripts\python.exe</code>
fastapi: command not found	activate the venv, or: <code>python -m uvicorn main:app --reload</code>
No module named 'app' (Lab 8)	<code>cd lab08_capstone/starter</code> (folder with <code>pytest.ini</code>)
address already in use	stop the old server (Ctrl+C) or use <code>--port 8002</code>
httpx2.ConnectError: connection refused	start the server in the other terminal
SyntaxError on <code>except*</code> / no TaskGroup	Python older than 3.11 - upgrade
<code>asyncio.run()</code> cannot be called from a running event loop	in Jupyter use: <code>await main()</code>
async but not faster	use <code>gather</code> / <code>TaskGroup</code> ; remove blocking calls
pip SSL / proxy errors	<code>pip install --proxy http://host:port ...</code> or internal mirror

Appendix C - asyncio cheat sheet

Also in CHEATSHEET.md

```
async def f(): ...           # coroutine function
asyncio.run(main())          # program entry, once
r = await f()                # run + wait
a, b = await asyncio.gather(f(), g()) # concurrent, ordered results
await asyncio.gather(*cs, return_exceptions=True) # best effort
t = asyncio.create_task(f()); r = await t # start now, collect later
async with asyncio.TaskGroup() as tg:    # structured concurrency (3.11+)
    tg.create_task(f())
except* ValueError as eg: ...           # handle ExceptionGroup
async with asyncio.timeout(2): await f() # TimeoutError (3.11+)
await asyncio.sleep(1)                 # never time.sleep in async
await asyncio.to_thread(blocking_fn, x) # blocking I/O -> thread
await loop.run_in_executor(proc_pool, cpu_fn, x) # CPU -> process
async with asyncio.Semaphore(5): ...    # cap concurrency
async with asyncio.Lock(): ...          # protect state across awaits
q = asyncio.Queue(maxsize=100); await q.put(x); x = await q.get(); q.task_done(); await q.join()
async with httpx2.AsyncClient(timeout=5) as c: r = await c.get(url)
asyncio.run(main(), debug=True)        # find slow/blocking steps
```

Appendix D - Further reading (official sources)



Python docs - asyncio

docs.python.org/3/library/asyncio.html (Coroutines & Tasks, Queues, Synchronization, Developing with asyncio)

FastAPI docs

fastapi.tiangolo.com/async/ · [/tutorial/background-tasks/](https://fastapi.tiangolo.com/tutorial/background-tasks/) · [/advanced/events/](https://fastapi.tiangolo.com/advanced/events/) (lifespan)

httpx2

github.com/pydantic/httpx2 (AsyncClient, timeouts, transports)

pytest-asyncio

pytest-asyncio.readthedocs.io (asyncio_mode, fixtures, 1.x migration)

Next steps

[Arq](#) / [Celery](#) / [Dramatiq](#) for durable jobs · [asyncpg](#) / [SQLAlchemy](#) async for databases · [tenacity](#) / [stamina](#) for retries

Appendix E - Trainer notes: v1 -> v2



Self-review of "009_Async Processing" and the improvements made in this edition

Issue found in v1	Improvement in this edition
Mixed Java/Spring and Python content; Python slide said "Use Spring's @Async"; Python step titled "Java/Spring Boot Path"	Python-only, consistent titles and objectives
Only FastAPI BackgroundTasks - asyncio itself (coroutines, event loop, gather, TaskGroup) was not taught	9 modules from first coroutine to capstone
"Exercise #2" with no #1; one lab; code shown as images (cannot copy)	8 progressive labs, starter + solution files, code as editable text
pip install "fastapi[all]"; no lifespan, testing or timeouts	fastapi[standard] + fastapi dev, lifespan, httpx2, pytest-asyncio 1.x, asyncio.timeout
Blocking-call trap (time.sleep in async def) not explained	Dedicated module + measurable FastAPI experiment
curl commands fail in Windows PowerShell; no setup/troubleshooting	Windows + macOS setup, client.py, Swagger UI, troubleshooting
Not self-explanatory: no expected outputs, no checkpoints	Expected output on every lab, quizzes with answers, speaker notes
Title text overlapping on several slides	Rebuilt layout, QA-checked for overflow

Now go make something fast.

Keep the lab kit. Re-run the labs in a week - the second time is where it sticks.

Questions? · Abinash Kumar Mishra